

ТЕОРИЯ ИСКУССТВЕННЫХ НЕЙРОННЫХ СЕТЕЙ

Каганов Ю.Т.

К.Т.Н., доцент каф. ИУ-9

**«Теоретическая информатика и компьютерные
технологии»**

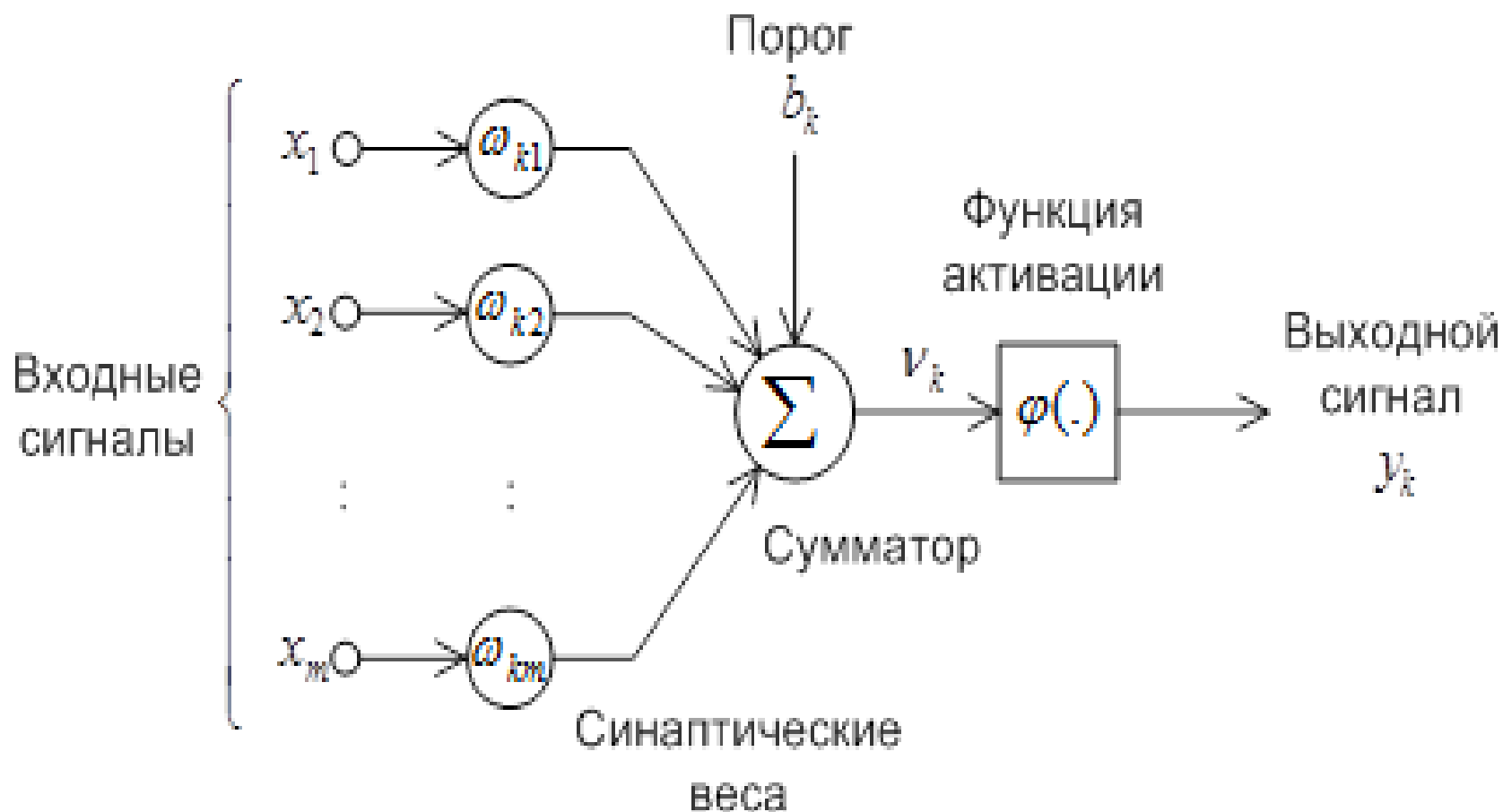
ЛЕКЦИЯ № 3

**Методы обучения ИНС и методы
оптимизации**

ПЛАН ЛЕКЦИИ

- 1. Модель искусственного нейрона.**
- 2. Функции активации.**
- 3. Архитектуры ИНС.**
 - 3.1. ИНС прямого распространения.**
- 4. Обучение нейронных сетей.**
 - 4.1. Парадигмы обучения.**
 - 4.2. Алгоритмы обучения.**
- 5. Элементы теории оптимизации.**
 - 5.1. Методы безусловной оптимизации.**
 - 5.2. Методы условной оптимизации**
- 6. Метрики оценки правильности**

Модель нейрона с произвольной функцией активации



Модель нейрона с сигмоидальной функцией активации

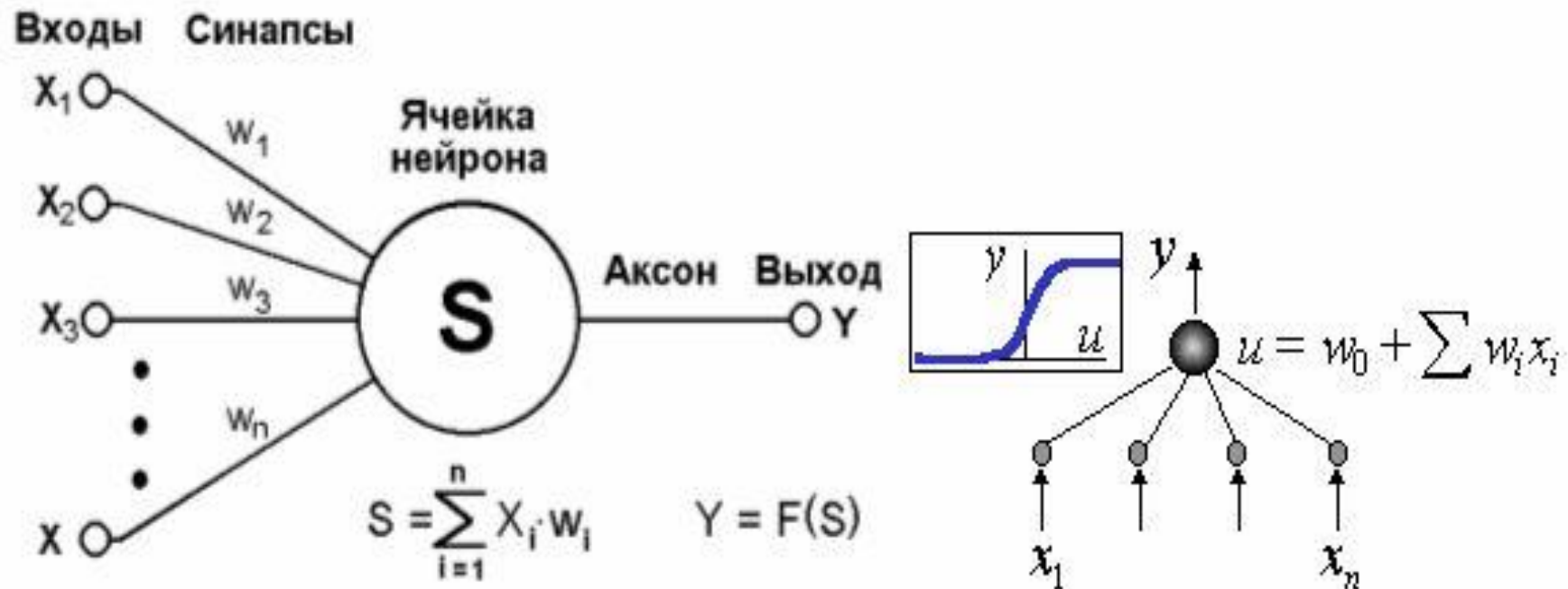
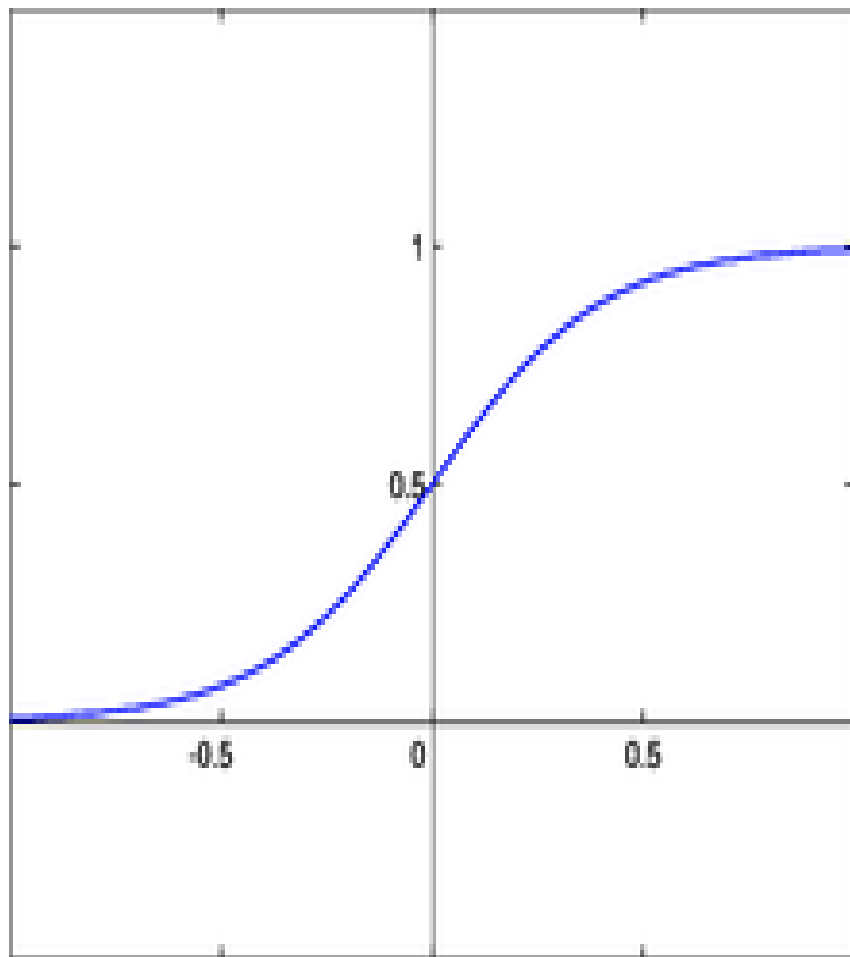


Рис. 1.2. Математическая модель нейрона

Сигмоидальная функция активации

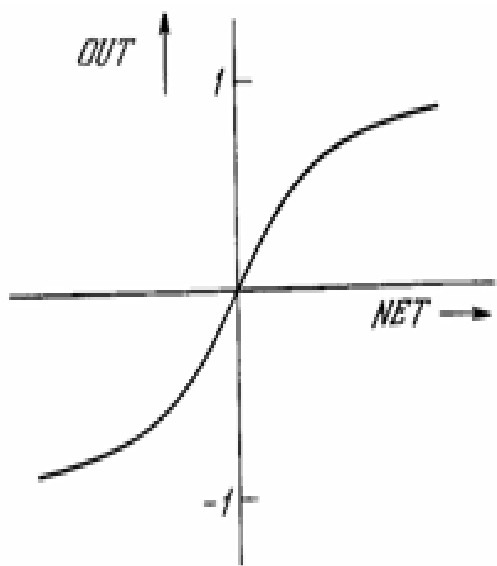


$$f(s) = \frac{1}{1 + e^{-as}}$$

$$S = \sum_{i=1}^n w_i x_i + b,$$

$$y = f(S),$$

Гиперболический тангенс

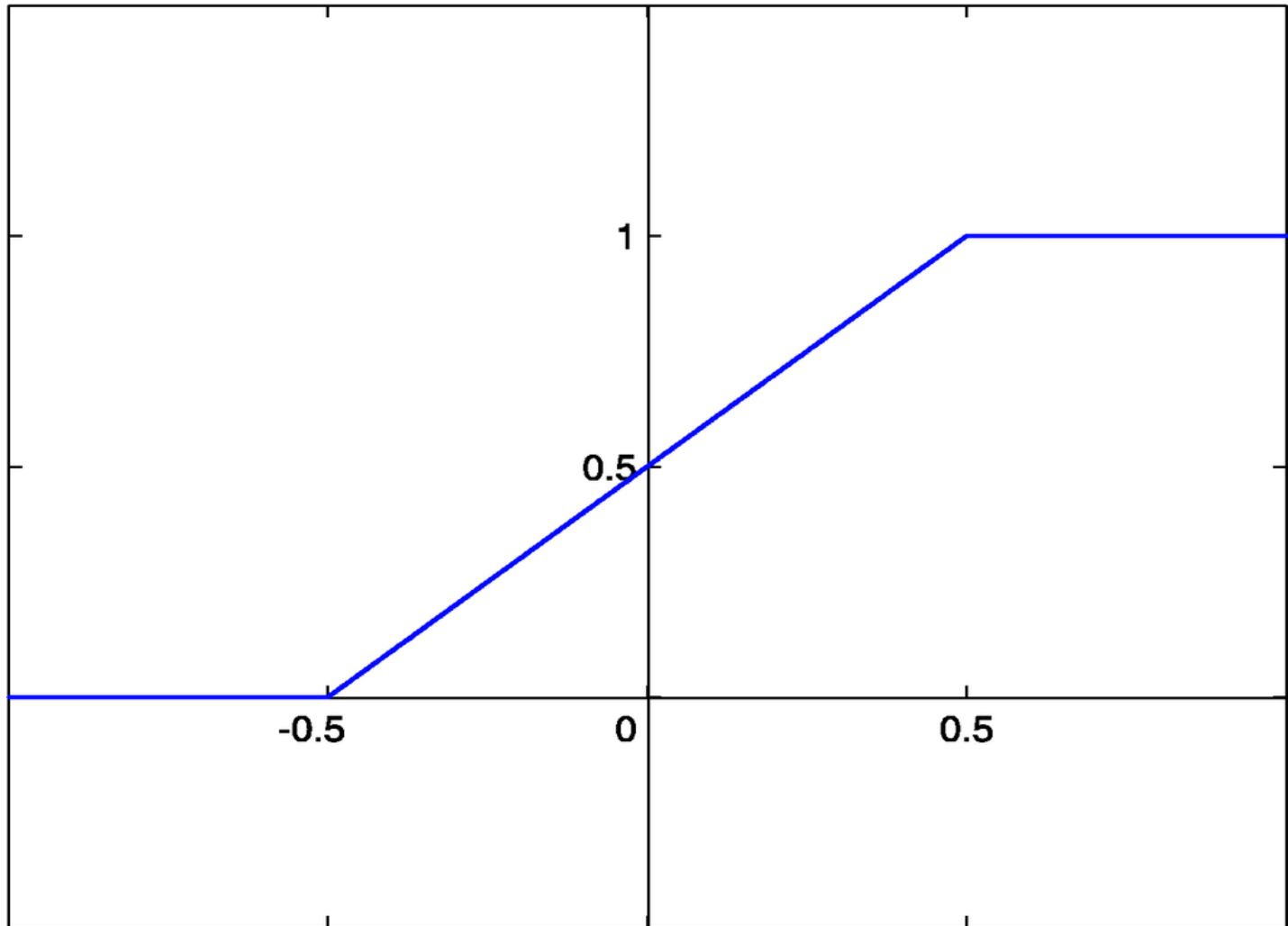


$$f(S) = \frac{e^{aS} - e^{-aS}}{e^{aS} + e^{-aS}}$$

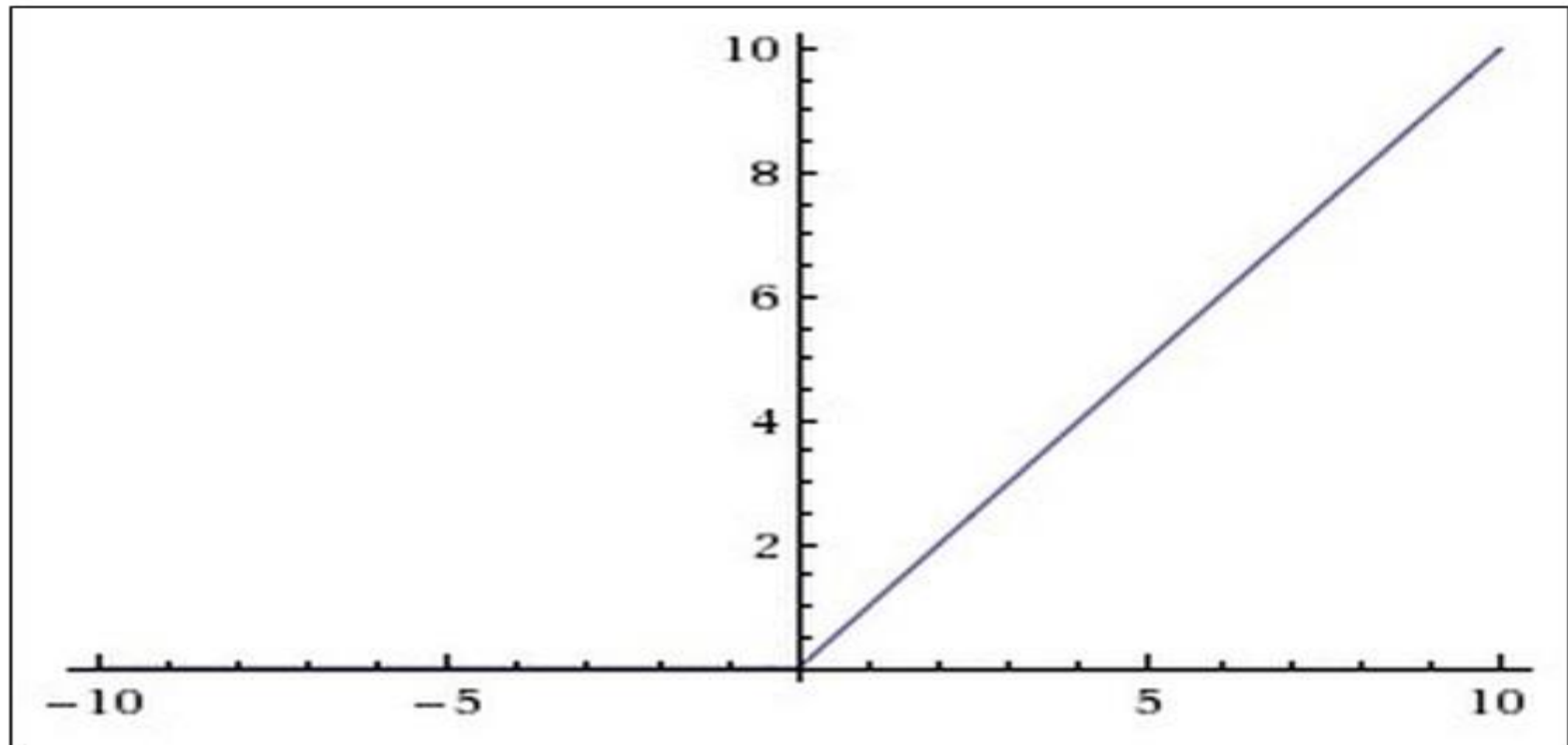
$$S = \sum_{i=1}^n w_i x_i + b,$$

$$y = f(S),$$

Линейная функция активации



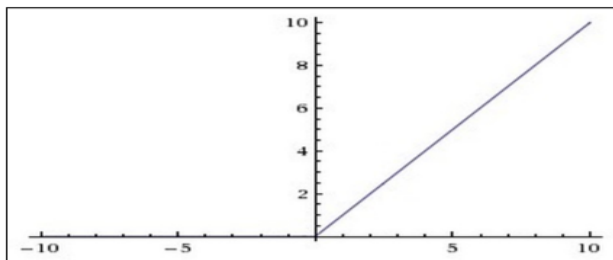
Функция активации ReLU



$$f(s) = \max(0, s)$$

Сравнение функций активации ReLU и гиперболического тангенса

ReLU (rectified linear unit)



$$f(s) = \max(0, s)$$

$$f'(s) = \begin{cases} 1, & s > 0 \\ \text{rand}(0.01, 0.05), & s \leq 0 \end{cases}$$

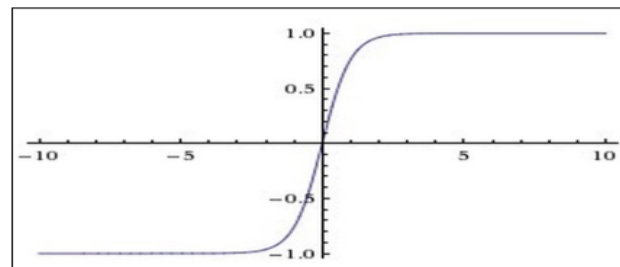
Достоинства

- лишена ресурсоемких операций
- отсекает ненужные детали
- отсутствует разрастание/затухание градиента
- быстрое обучение

Недостатки

- не всегда надежна, в процессе обучения может "умирать"
- сильно зависима от инициализации весов

Гиперболический тангенс



$$f(s) = \frac{e^{2s} - 1}{e^{2s} + 1}$$

$$f'(s) = 1 - f(s)^2$$

Достоинства

- простое вычисление производной через значение своей функции
- область значений от -1 до 1

Недостатки

- затухание или увеличение градиента
- ресурсоемкая по сравнению с ReLU

Архитектуры нейронных сетей

Существует многообразие архитектур

A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

- Backfed Input Cell
- Input Cell
- Noisy Input Cell
- Hidden Cell
- Probabilistic Hidden Cell
- Spiking Hidden Cell
- Output Cell
- Match Input Output Cell
- Recurrent Cell
- Memory Cell
- Different Memory Cell
- Kernel
- Convolution or Pool

Perceptron (P)



Feed Forward (FF)



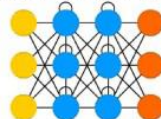
Radial Basis Network (RBF)



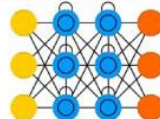
Deep Feed Forward (DFF)



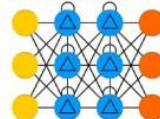
Recurrent Neural Network (RNN)



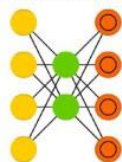
Long / Short Term Memory (LSTM)



Gated Recurrent Unit (GRU)



Auto Encoder (AE)



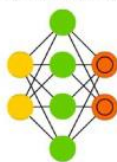
Variational AE (VAE)



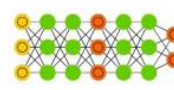
Denising AE (DAE)



Sparse AE (SAE)



Generative Adversarial Network (GAN)



Liquid State Machine (LSM)



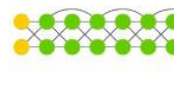
Extreme Learning Machine (ELM)



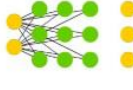
Echo State Network (ESN)



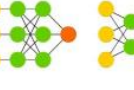
Deep Residual Network (DRN)



Kohonen Network (KN)



Support Vector Machine (SVM)



Neural Turing Machine (NTM)



Markov Chain (MC)



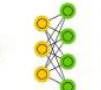
Hopfield Network (HN)



Boltzmann Machine (BM)



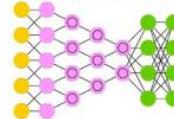
Restricted BM (RBM)



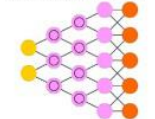
Deep Belief Network (DBN)



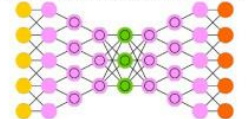
Deep Convolutional Network (DCN)



Deconvolutional Network (DN)



Deep Convolutional Inverse Graphics Network (DCIGN)

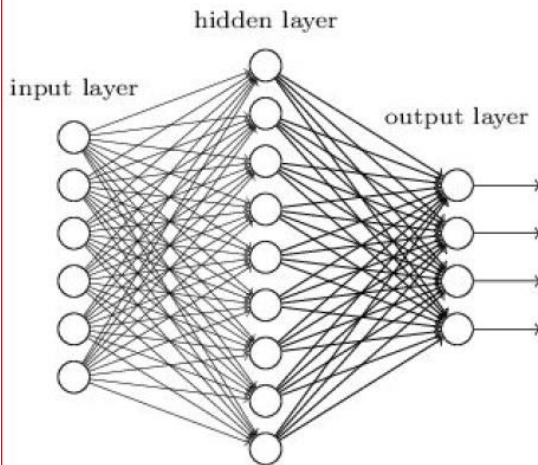


<http://www.asimovinstitute.org/neural-network-zoo/>

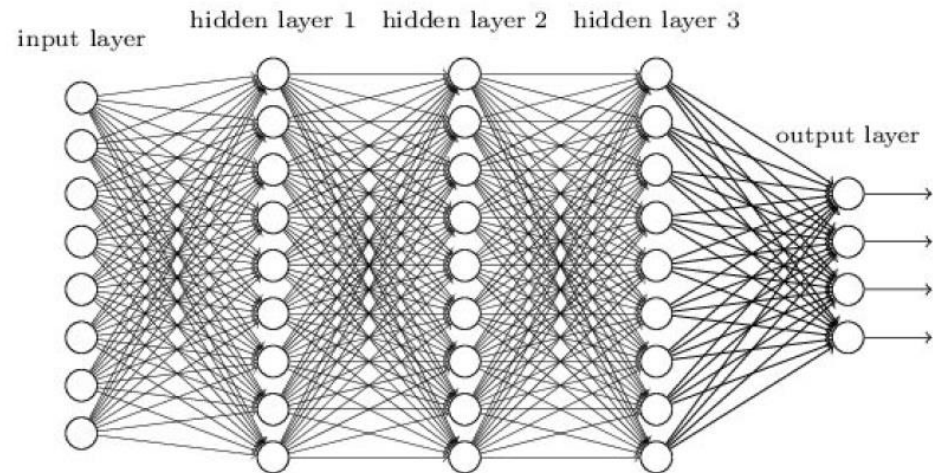
Нейронные сети прямого распространения

Искусственная нейросеть

"Non-deep" feedforward neural network



Deep neural network



Обучение нейронных сетей

- Существует универсальный *путь* обучения нейронных сетей - *минимизация* оценки как *неявно заданной функции* параметров сети.

При реализации этого подхода предполагается, что:

задана обучающая выборка, состоящая из векторов входных сигналов;

известны требования к соответствующим выходным сигналам, зафиксированные в функции оценки E ;

оценка E по всей выборке или какой-либо ее части строится известным способом по значениям E .

После подготовки (создание обучающей выборки, выбор функции оценки E , предобработка входных данных и т.п.), предшествующей обучению, имеем способ вычисления некоторой функции E , *минимизация* которой как функции параметров позволяет настроить *сеть* для правильной работы.

Парадигмы обучения нейронных сетей

- 1. Обучение с учителем.**
- 2. Обучение без учителя.**

Существует две парадигмы обучения нейронных сетей – с учителем и без учителя. В первом случае, на входной вектор имеется готовый ответ, во втором случае нейронная сеть самообучается. У каждого вида обучения есть своя ниша задач и по большому счету они не пересекаются. На данный момент придумано и запатентовано большое количество архитектур нейронных сетей и методов их обучения. Но основными (исходными) являются – для обучения с учителем это «алгоритм обратного распространения ошибки», а для обучения без учителя это алгоритмы Хебба и Кохонена.

Обучение с учителем

d_i – определяемая учителем цель (метка) на выходном нейроне сети

$y_i(w)$ – получаемая в результате работы нейронной сети реальное значение

$e_i(w) = (y_i(w) - d_i)$ – величина ошибки (потери) на текущем этапе работы сети

$$E(w) = \frac{1}{2} \sum_{ij} [y_{ij}(w) - d_{ij}]^2 - \text{общая ошибка по всем выходным нейронам}$$

Обратное распространение ошибки определяется коррекцией весов w_{ij} в связи с уменьшением ошибки в процессе обучения:

$$w_{ij}(t) = w_{ij}(t - 1) + \Delta w_{ij}(t - 1).$$

Основная проблема состоит в выборе $\Delta w_{ij}(t - 1)$.

Выбор приращения $\Delta w_{ij}(t - 1)$ осуществляется на основе методов оптимизации, т.е. путем минимизации функции ошибки.

$$\nabla E(w) = \frac{DE(w)}{Dw} = 0$$

Наиболее распространенным классом методов является класс методов математического программирования, основанных на градиентных методах.

Методы оптимизации

- Оптимизационные задачи заключаются в нахождении минимума (максимума) заданной функции. Такую функцию называют целевой. Как правило, целевая функция — сложная функция, зависящая от некоторых входных параметров. В оптимизационной задаче требуется найти значения входных параметров, при которых целевая функция достигает минимального (максимального) значения. Существует целый класс оптимизационных методов. Условно все оптимизационные методы можно разделить на методы, использующие понятие производной (градиентные методы) и стохастические методы (например, методы группы Монте-Карло). С их помощью можно найти экстремальное значение целевой функции, но не всегда можно быть уверенным, что получено значение глобального экстремума. Нахождение локального экстремума вместо глобального называется преждевременной сходимостью. Помимо проблемы преждевременной сходимости существует другая проблема — время процесса вычислений. Зачастую более точные оптимизационные методы работают очень долго.

Постановка задачи оптимизации

$$\begin{cases} x^e = \mathit{Arg} \min f(x), \\ x_i \geq 0, i = 1..n, \\ g_j(x) \leq 0, j = 1..m. \end{cases}$$

Целевые функции

1. Функция ошибки

$$F(w) = \sum_i L(a(x_i, w), y_i) = E(w) = \frac{1}{n} \sum_{ij} [y_{ij}(w) - d_{ij}]^2$$

2. Расстояние Кульбака-Лейблера

$$KL(P|Q) = \int p(x) \log \frac{p(x)}{q(x)} dx \quad KL(P|Q) = \sum_i p(x) \log \frac{p(x)}{q(x)}$$

3. Перекрестная энтропия

$$H(p, q) = E_p[-\log(q)] = - \sum_y p(y) \log(q(y))$$

Методы математического программирования

- Методы математического программирования основаны на использовании итерационных алгоритмов поиска значений переменных, отвечающих экстремуму целевой функции при заданных ограничениях. При этом строится алгоритмическое отображение над функцией спуска, включающей как целевую функцию, так и функции ограничений.

Стратегия методов спуска

$$\text{ЗВП} \quad \left\{ \begin{array}{l} x^e = \text{Arg min } f(x), x \in X; \\ x_i \geq 0, i = 1, \dots, n; \\ g_j(x) \leq 0, j = 1, \dots, m. \end{array} \right.$$

$$x^e, \bar{x} \in \Omega_\varepsilon, \bar{x} = \lim_{k \rightarrow N} x^k$$

Алгоритмическое отображение состоит в том, что требуется на каждой итерации производить поиск $x^{(k+1)} = A\{\varphi[f(x^{(k)}), g(x^{(k)})]\}$ так, чтобы

$$\varphi(x^{(k+1)}) \leq \varphi(x^{(k)})$$

$$\|x^e - \bar{x}\| \leq \delta \quad x^e, \bar{x} \in \Omega_\varepsilon$$

$$|f(x^e) - f(\bar{x})| \leq \varepsilon$$

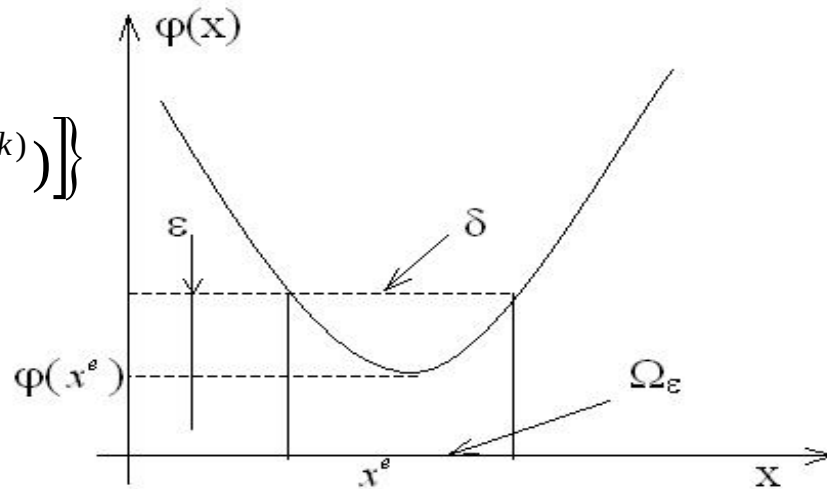
Выпуклая функция:

$$\varphi(x) = \varphi[f(x), g(x)]$$

$$\varphi : X \rightarrow R$$

$$X \subset E^n$$

$$\Omega_\varepsilon \subseteq U_\varepsilon(x) \subset E^n$$



Стратегия методов спуска

Стратегия ММП для ЗВП состоит в выборе $x^{(k+1)} = x^{(k)} + \Delta x^{(k)}$

таким образом, чтобы $\varphi(x^{(k)} + \Delta x^{(k)}) \leq \varphi(x^{(k)})$ для $\forall x^{(k)} \in X$

При этом для большинства методов МП $\Delta x^{(k)} = \alpha^{(k)} d^{(k)}$,

где $d^{(k)} \in E^n$ – направление поиска, $\alpha^{(k)} \in E^1$ – шаг в направлении поиска.

Решение задачи ММП состоит в следующем:

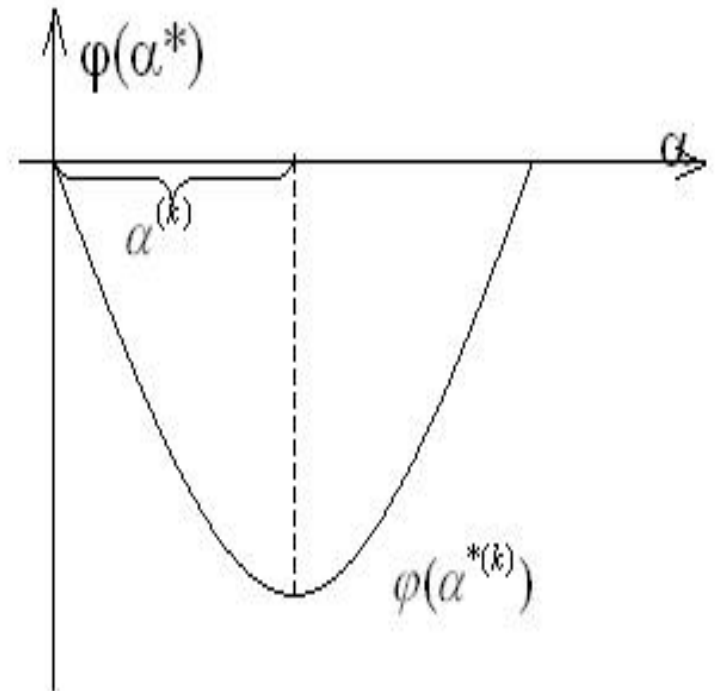
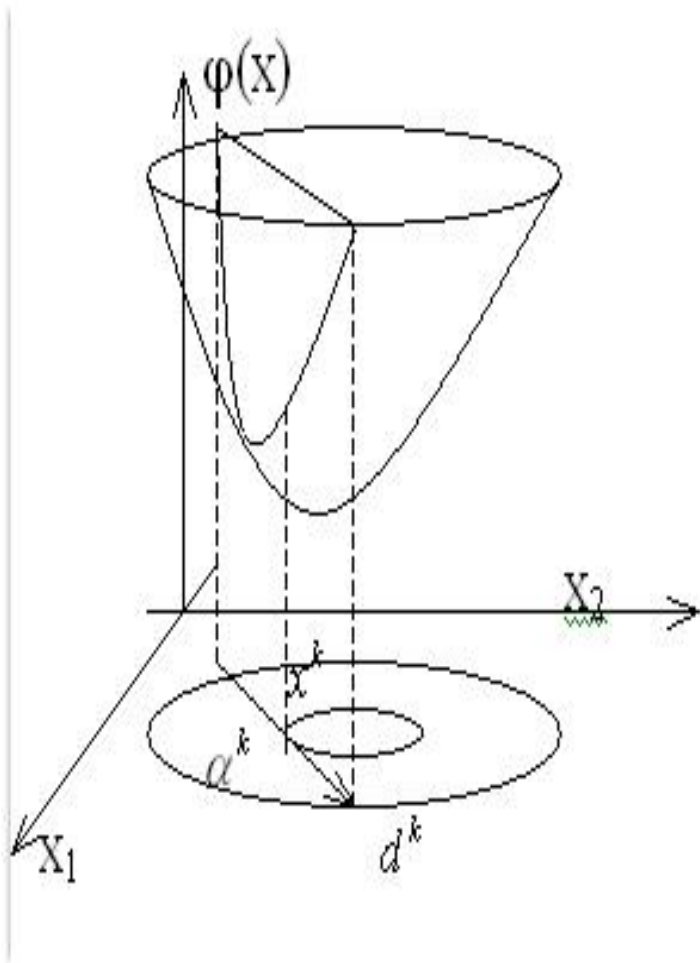
1. Выбор направления поиска на каждой итерации

оптимизации $d^{(k)} \in X \subset E^n$

2. Выбор величины шага $\alpha^{*(k)} \in E^1$

таким образом, чтобы $\alpha^{*(k)} = \text{Arg min } \varphi(x^{(k)} + \alpha^{(k)} d^{(k)})$ в направлении вектора $d^{(k)} \in X \subset E^n$

Стратегия методов спуска



Простейшие методы ММП для решения ЗВП

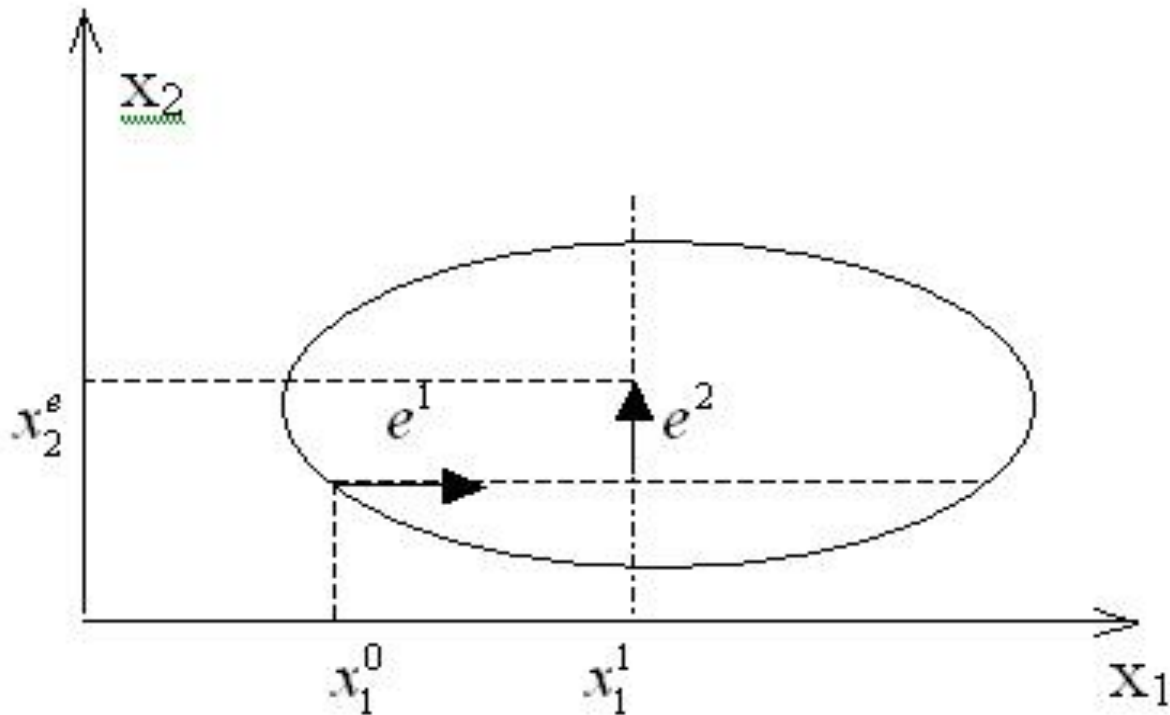
Их можно разбить на 3 основных класса:

1. Методы 0-го порядка (не использующие производные).
2. Методы 1-го порядка (использующие 1-е производные).
3. Методы 2-го порядка (использующие 2-е производные).

Рассмотрим простейшие представители каждого из этих классов.

Метод покоординатного спуска (метод 0-го порядка)

Функция $\varphi(x) \in C^0(X)$ — непрерывная функция.



Алгоритм:

Ш1. Направление d^k - выбирается вдоль координаты X_k от точки $x^{(k)}$, т.е. $d^{(k)} = \frac{x_k}{\|x_k\|} = e^k$

Ш2. Определяется величина шага

$$\alpha^{*(k)} = \text{Arg min}_{\alpha \in E^1} \varphi(x^{(k)} + \alpha^{(k)} d^{(k)})$$

Ш3. Формируется новое значение

$$x^{(k+1)} = x^{(k)} + \alpha^{*(k)} d^{(k)}$$

Ш4. Проверка сходимости:

Если $(|\varphi(x^{(k+1)}) - \varphi(x^{(k)})| \leq \varepsilon, \|x^{(k+1)} - x^{(k)}\| \leq \delta)$, то *КОНЕЦ*

Иначе $k=k+1$ и переход на Ш 1.

Достоинства: возможность оптимизации для задач с недифференцируемыми функциями.

Недостатки:

1. Низкая скорость сходимости (сублинейная, т.е. $\gamma < 1$)

2. Возможность закливания при плохой обусловленности задачи, т.е. когда функция $\varphi(x)$ имеет овражный характер.

Обусловленность задачи

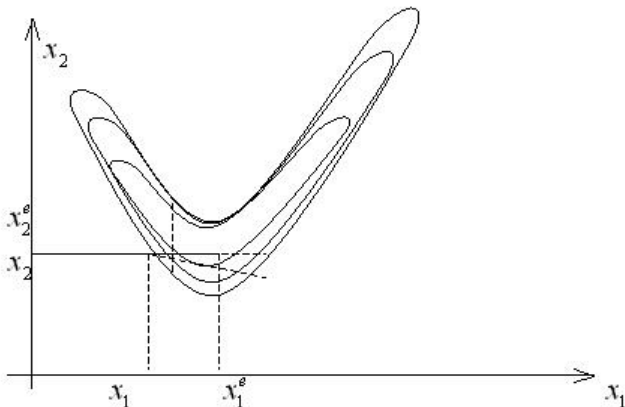
Определяется числом обусловленности матрицы Гессе (если она существует), т.е. отношением $\chi = \frac{L}{l}$.

Где L - максимальное собственное число $H(x^{(k)})$

l — минимальное собственное число $H(x^{(k)})$.

В случае отсутствия матрицы Гессе числом обусловленности может быть отношение:

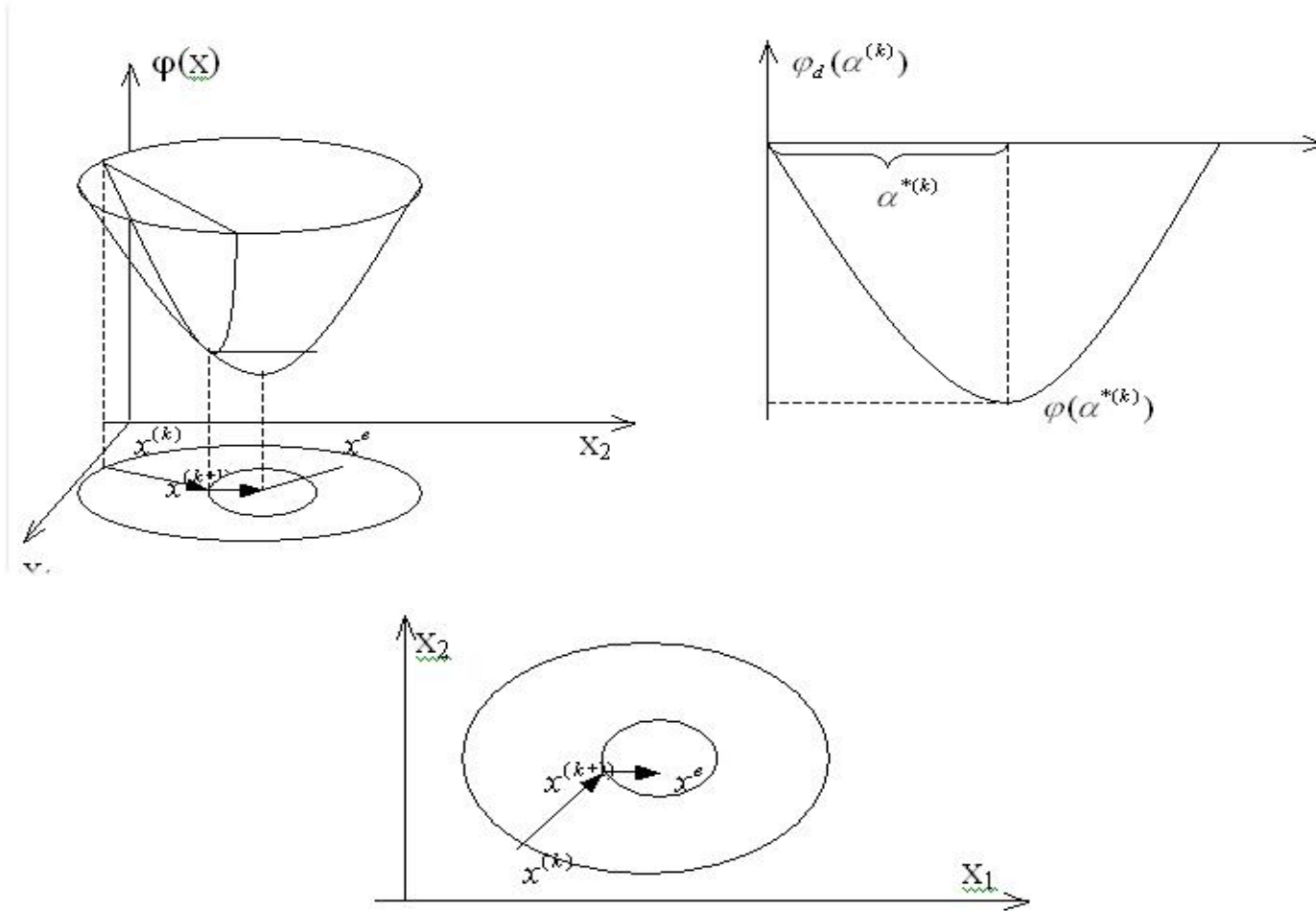
$$\chi = \lim_{\delta \rightarrow 0} \frac{\sup \|x^{(k)} - x^e\|^2}{\inf \|x^{(k)} - x^e\|^2} .$$



Пример функция Розенброка :

$$f(x_1, x_2) = 100(x_2^2 - x_1)^2 + (x_2 - 1)^2$$

Метод наискорейшего спуска (градиентный метод)



Классы градиентных методов

Возможно 2 основных подхода к реализации градиентного спуска:

- *Пакетный (batch)*, когда на каждой итерации обучающая выборка просматривается целиком, и только после этого изменяется. Это требует больших вычислительных затрат.
- *Стохастический (stochastic/online)*, когда на каждой итерации алгоритма из обучающей выборки каким-то (случайным) образом выбирается только один объект (или несколько объектов). Таким образом вектор w настраивается на каждый вновь выбираемый объект или серию объектов.

Метод наискорейшего спуска (метод 1-го порядка)

Функция $\varphi(x) \in C^1(X)$ - класс 1 раз дифференцируемых функций.

$x^{(k+1)} = x^{(k)} + \Delta x^{(k)} = x^{(k)} + \alpha^{(k)} d^{(k)}$ – в ряд Тейлора, тогда, если учитывать
Разложим функцию $\varphi(x^{(k)} + \alpha^{(k)} d^{(k)})$ только 2 члена ряда, получим:

$$\varphi(x^{(k+1)}) = \varphi(x^{(k)}) + \nabla^T \varphi(x^{(k)}) \Delta x^{(k)} + o(\|\Delta x^{(k)}\|)$$

Для того, чтобы функция уменьшалась, необходимо, чтобы $d^{(k)} \sim -\nabla \varphi(x^{(k)})$
 $d^{(k)} = -\frac{\nabla \varphi(x^{(k)})}{\|\nabla \varphi(x^{(k)})\|}$. Тогда $x^{(k+1)} = x^{(k)} - \frac{\nabla \varphi(x^{(k)})}{\|\nabla \varphi(x^{(k)})\|} \alpha^{*(k)}$

При этом $\alpha^{*(k)} = \text{Arg min}_{x \in E^1} \varphi(x^{(k)} + \alpha^{*(k)} d^{(k)})$

Особенность МНС: $\nabla^T \varphi(x^{(k+1)}) \nabla \varphi(x^{(k)}) = 0$; то есть $d^{(k+1)T} d^{(k)} = 0$.

Достоинства. Линейная скорость сходимости для функций $\varphi(x)$ близких к квадратичным.

Недостатки:

1. Функция $\varphi(x) \in C^1(X)$ – должна быть дифференцируема.
2. Возможность закливания при плохой обусловленности задачи.

Метод Ньютона (метод второго порядка)

Условием использования этого метода

является: $\varphi(x) \in C^2(X)$,

а также условие выпуклости $\varphi(x)$ в окрестности точки поиска.

Точка x^* – стационарная точка (точка локального экстремума)

Пусть: $\varphi(x^*) = \min(\varphi(x))$ Тогда :

$$\bar{x} \in U_\varepsilon(x^*) \quad \varphi(x^*) = \varphi(\bar{x} + \Delta x) = \varphi(\bar{x}) + \nabla^T \varphi(x) \Delta x^* + \frac{1}{2} \Delta x^T H(\bar{x}) \Delta x + o(\|\Delta x\|^2)$$

$$x^* = \bar{x} + \Delta x \quad \text{где} \quad H(\bar{x}) = \left\{ \frac{\partial^2 \varphi(\bar{x})}{\partial x_i \partial x_j} \right\}_{i=1..n, j=1..n}$$

Разложение функции $\varphi(x^*)$ в окрестности x^* в ряд Тейлора до 3-го члена дает квадратичную аппроксимацию этой функции в $U_\varepsilon(x^*)$:

$$0 = \nabla \varphi(\bar{x}) + H^T(\bar{x})(x^* - \bar{x}) \quad . \quad x^{(k+1)} = x^{(k)} - H^{-1}(x^{(k)}) \nabla \varphi(x^{(k)}) .$$

$$d^{(k)} = -H^{-1}(x^{(k)}) \nabla \varphi(x^{(k)}) ; \quad \alpha^{*(k)} = \text{Arg min } \varphi_d(\alpha^{(k)}) ; \quad x^{(k+1)} = x^{(k)} + \alpha^{*(k)} d^{(k)} .$$

Метод Ньютона

(метод второго порядка)

Сопряженность: $d^{(k+1)} H^{-1}(x^{(k)}) d^{(k)} = 0$

Достоинства :

Для функций, близких к квадратичным, скорость сходимости алгоритма суперлинейная.

Недостатки:

- 1.Необходимость вычисления $H^{-1}(x^{(k)})$.
- 2.Процесс решения сильно зависит от выбора $x^{(0)}$ (т.е. от начальной точки)
- 3.Функция $\varphi(x)$ должна быть выпуклой во всех точках $x^{(k)}$, то есть $\det H(x^k) > 0$.

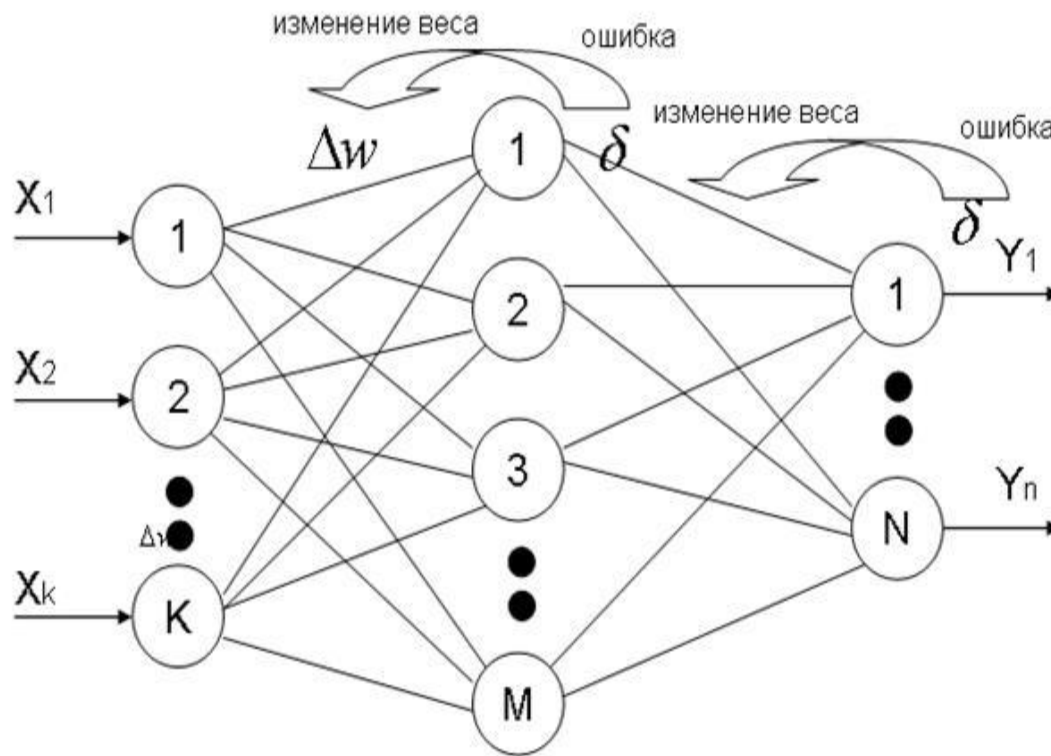
Градиентный спуск

- Метод градиентного спуска с некоторой модификацией широко применяется для обучения перцептрона. Он известен как метод обратного распространения ошибки. При обучении нейросети типа «персептрон» требуется изменять весовые коэффициенты сети так, чтобы минимизировать среднюю ошибку на выходе нейронной сети при подаче на вход последовательности обучающих входных данных. Формально, чтобы сделать всего один шаг по методу градиентного спуска (сделать всего одно изменение параметров сети), необходимо подать на вход сети последовательно абсолютно весь набор обучающих данных, для каждого объекта обучающих данных вычислить ошибку и рассчитать необходимую коррекцию коэффициентов сети (но не делать эту коррекцию), и уже после подачи всех данных рассчитать сумму в корректировке каждого коэффициента сети (сумма градиентов) и произвести коррекцию коэффициентов «на один шаг».

Стохастический градиентный спуск

- Пакетный градиентный спуск вычисляет один шаг, используя сразу весь набор данных, тогда как стохастический за шаг использует только 1 элемент. Можно два этих варианта скрестить, получив мини-пакетный (mini-batch) спуск, который за раз обрабатывает, к примеру, 100 элементов, а не все или один.
- На 200 элементах разницы в скорости почти никакой нет, однако, увеличив количество элементов до 2000 (что тоже очень мало) и подкорректировав скорость обучения, стохастический вариант оказывается значительно более эффективным чем пакетный режим.
- Однако, в силу стохастической природы, иногда метод промахивается, осциллируя возле минимума без возможности остановиться.

Алгоритм обратного распространения ошибки



- В этом алгоритме происходит распространение ошибки от выходов НС ко входам, то есть в направлении обратном распространению сигналов обычном режиме работы. Согласно методу наименьших квадратов, минимизируемой целевой функцией ошибки НС является величина:

$$E(w) = \frac{1}{2} \sum_{j,p} (y_{j,p}^{(M)} - d_{j,p})^2$$

Алгоритм Stochastic Gradient Descent (SGD)

Вход:

- X_j — обучающая выборка
- η — темп обучения
- λ — параметр сглаживания функционала

Выход:

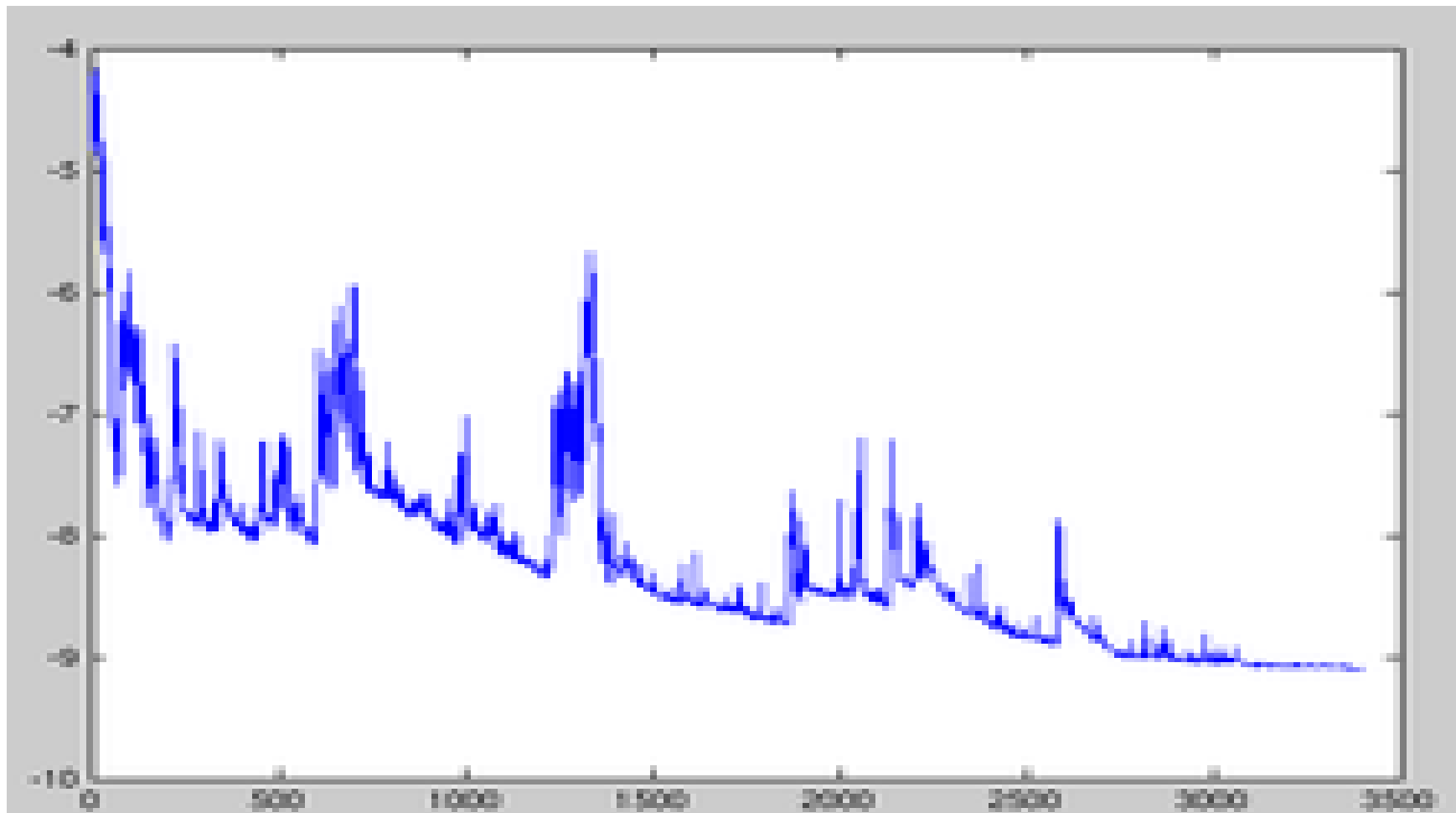
- $W = [w_1, w_2, \dots, w_N]^T$ — вектор весов

Алгоритм Stochastic Gradient (SG)

- **Тело:**

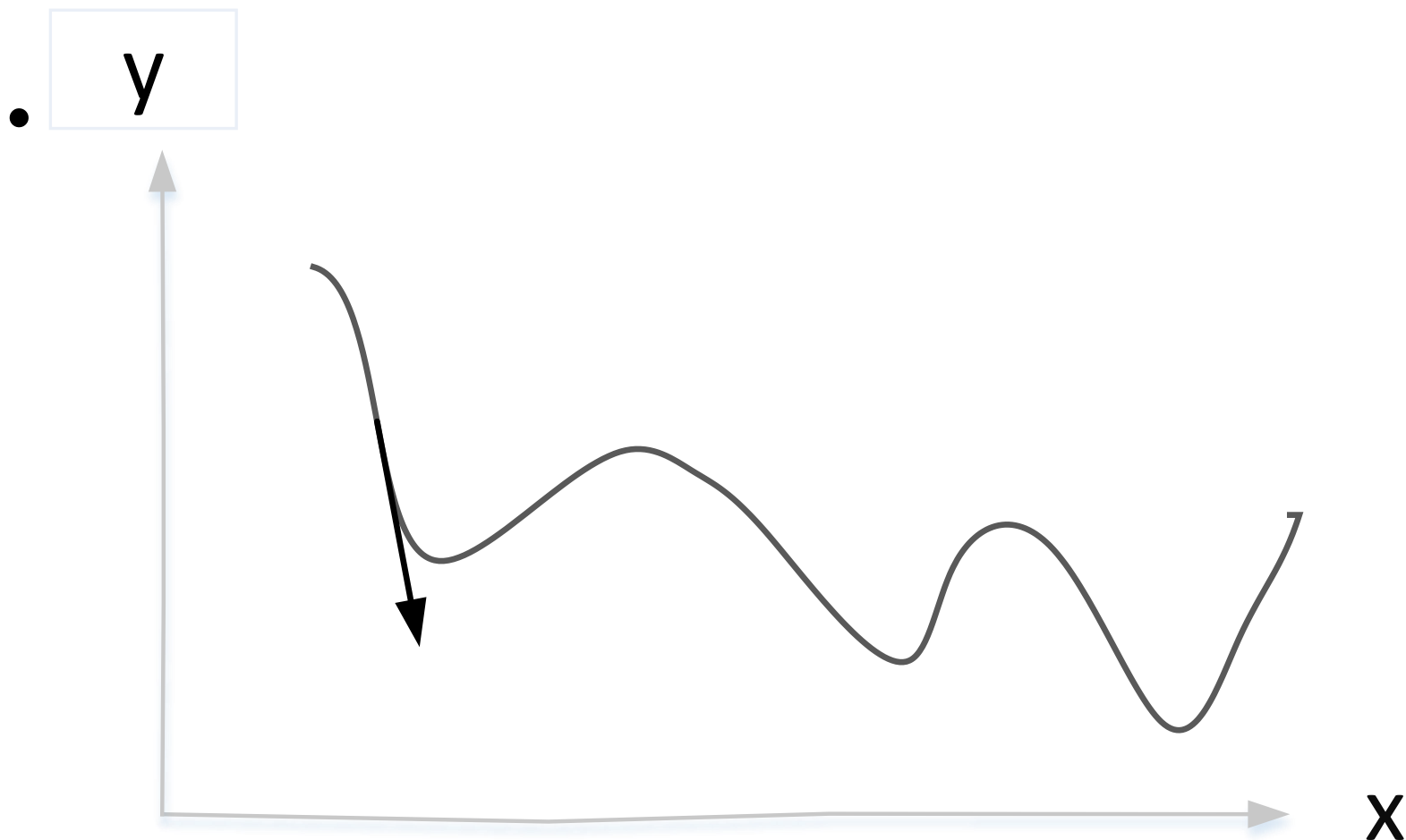
1. Инициализировать веса: $w_j 0$ ($j = 0, 1, \dots, N$);
2. Инициализировать текущую оценку; $F(w) = \sum_i L(a(x_i, w), y_i)$
3. Повторить:
 - a) выбрать объект x_i из X^l (случайным образом);
 - b) вычислить выходные значения алгоритма $a(x_i, w)$ и ошибку $E_i = L(a(x_i, w), y_i)$;
 - c) Сделать шаг градиентного спуска:
$$w(t) = w(t - 1) - \eta \cdot \nabla_a L(a(x_i, w), y) \cdot \nabla_w \varphi(\langle w, x_i \rangle) x_i;$$
 - d) Оценить значение функционала:
$$Q(t) = (1 - \lambda)Q(t - 1) + \lambda E(t - 1);$$
4. Пока значения Q не стабилизируются и/или веса W не перестанут изменяться.

Стохастический градиентный спуск



Флуктуации целевой функции по мере работы алгоритма

Функция ошибки (целевая функция)



$Y=f(x)$ - Многоэкстремальная функция

Алгоритм обратного распространения

- Согласно методу наименьших квадратов, минимизируемой целевой функцией ошибки НС является величина:

$$E(w) = \frac{1}{2} \sum_{j,p} (y_{j,p}^{(N)} - d_{j,p})^2$$

$$\Delta w_{ij}^{(n)} = -\eta \cdot \frac{\partial E}{\partial w_{ij}} \quad \frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_j} \cdot \frac{dy_j}{ds_j} \cdot \frac{\partial s_j}{\partial w_{ij}}$$

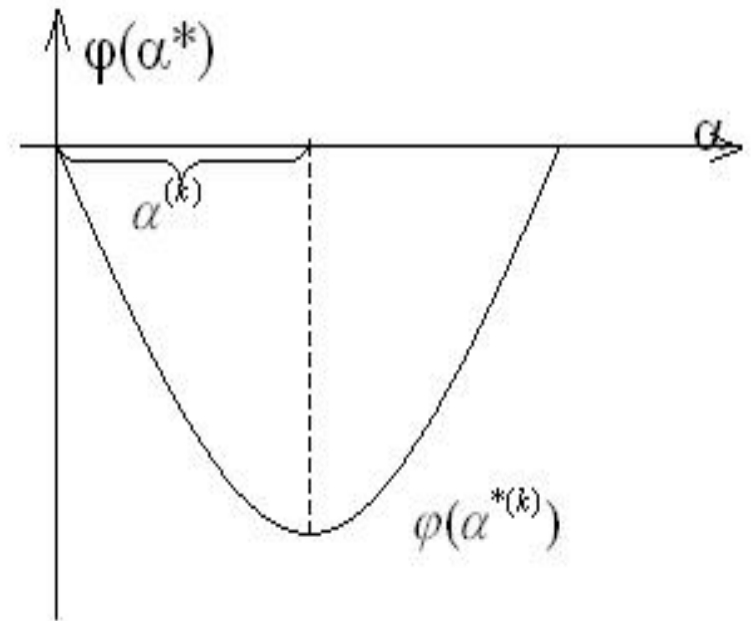
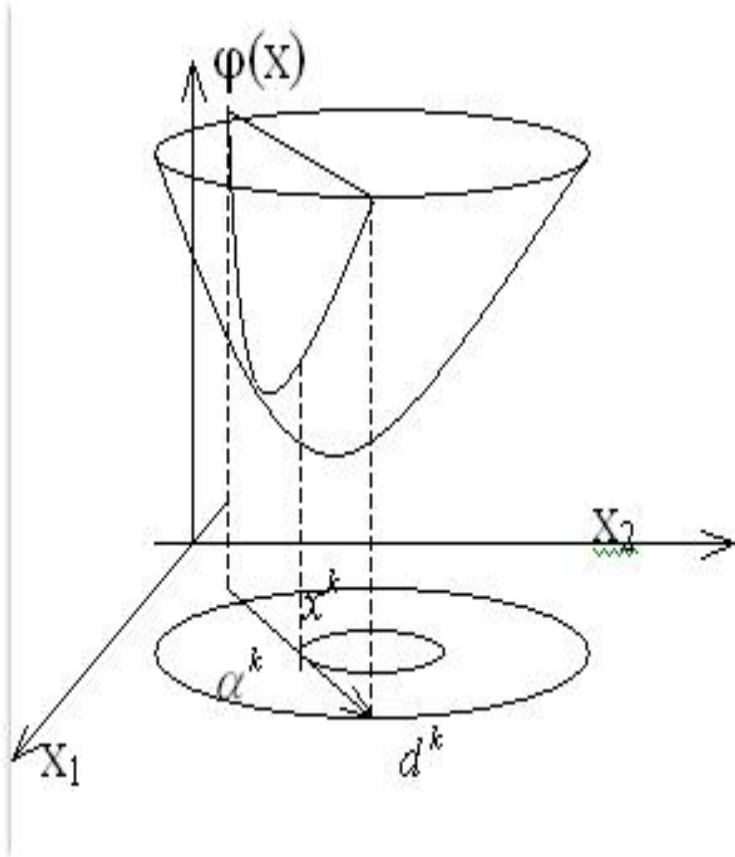
$$\delta_l^{(N)} = (y_l^{(N)} - d_l) \cdot \frac{dy_l}{ds_l} \quad \frac{\partial E}{\partial y_j} = \sum_k \frac{\partial E}{\partial y_k} \cdot \frac{dy_k}{ds_k} \cdot \frac{\partial s_k}{\partial y_j} = \sum_k \frac{\partial E}{\partial y_k} \cdot \frac{dy_k}{ds_k} \cdot w_{jk}^{(n+1)} \quad \delta_j^{(n)} = \frac{\partial E}{\partial y_j} \cdot \frac{dy_j}{ds_j}$$

$$\Delta w_{ij}^{(n)} = -\eta \cdot \delta_j^{(n)} \cdot y_i^{(n-1)}$$

Иногда для придания процессу коррекции весов некоторой инерционности, сглаживающей резкие скачки при перемещении по поверхности целевой функции, дополняется значением изменения веса на предыдущей итерации

$$\Delta w_{ij}^{(n)}(t) = -\eta \cdot (\mu \cdot \Delta w_{ij}^{(n)}(t-1) + (1-\mu) \cdot \delta_j^{(n)} \cdot y_i^{(n-1)})$$

Итеративный процесс поиска экстремума



Методы условной оптимизации

- Рассмотрим задачу математического программирования в общем случае, т. е. при наличии ограничений типа равенств и неравенств.
- Для решения таких задач существует широкий спектр методов нелинейного программирования. Среди них выделяют 2 основных класса:
- **Прямые методы и двойственные методы.**
- Двойственные методы наиболее часто используемые методы в обучении нейронных сетей. Поэтому мы остановимся на изложении именно этих методов.

Двойственные методы.

- Среди этих методов наиболее широко используются:
 - Методы штрафных и барьерных функций.
 - Методы модифицированных функций Лагранжа.
 - Методы спроектированного лагранжиана.
- Все эти методы основаны на использовании двойственной функции – аналог функции Лагранжа.
- Общий принцип двойственных методов заключается в замене решения исходной задачи на решение последовательности экстремальных задач без ограничений. При этом функцией спуска, как правило является аналог функций Лагранжа. Эти методы в сочетании с прямыми оказались чрезвычайно эффективными для решения задач с нелинейными функциями. В настоящее время среди двойственных методов наиболее эффективными являются методы модифицированных функций Лагранжа.

Постановка задачи оптимизации

$$\begin{cases} x^e = \mathit{Arg} \min f(x), \\ x_i \geq 0, i = 1..n, \\ g_j(x) \leq 0, j = 1..m. \end{cases}$$

Функция Лагранжа (лагранжиан)

$$L(x, \mu) = f(x) + \sum_{j=1}^m \mu_j g_j(x)$$

Методы штрафных функций

- Построим вспомогательную функцию, аналогичную лагранжиану:

$$\Phi^{(s)}(x, p) = f(x) + P^{(s)} \sum_{j=1}^l \varphi_j[g_j(x)] + P^{(s)} \sum_{j=l+1}^m \psi_j[g_j(x)]$$

$$x^{e(1)} = \text{Arg min } \Phi(x, p^{(1)})$$

$$x^{e(2)} = \text{Arg min } \Phi(x, p^{(2)})$$

•

•

•

$$x^{e(s)} = \text{Arg min } \Phi(x, p^{(s)})$$

Метрики оценки правильности

Predicted class		
Positive	Negative	
TP	FN	Positive
FP	TN	Negative
		True class

Correct/incorrect prediction?

True	False
False	True

Meaning of true and false

Positive/negative class prediction?

+	-
+	-

Meaning of positive and negative

- Класс, который представляет для нас интерес, называется «положительным», а оставшийся — «отрицательным».
- Заметим, что для каждого объекта в выборке возможно 4 ситуации:
- мы предсказали *положительную* метку и *угадали*. Будет относиться такие объекты к **true positive (TP)** группе. True — потому что предсказали мы правильно, а positive — потому что предсказали положительную метку;
- мы предсказали *положительную* метку, но *ошиблись* в своём предсказании — **false positive (FP)**. False, потому что предсказание было неправильным;
- мы предсказали *отрицательную* метку и *угадали* — **true negative (TN)**;
- и наконец, мы предсказали *отрицательную* метку, но *ошиблись* — **false negative (FN)**. Для удобства все эти 4 числа изображают в виде таблицы, которую называют **confusion matrix (матрицей ошибок)**

Вычисляя долю недиагональных элементов, мы приходим к метрике **error rate**:

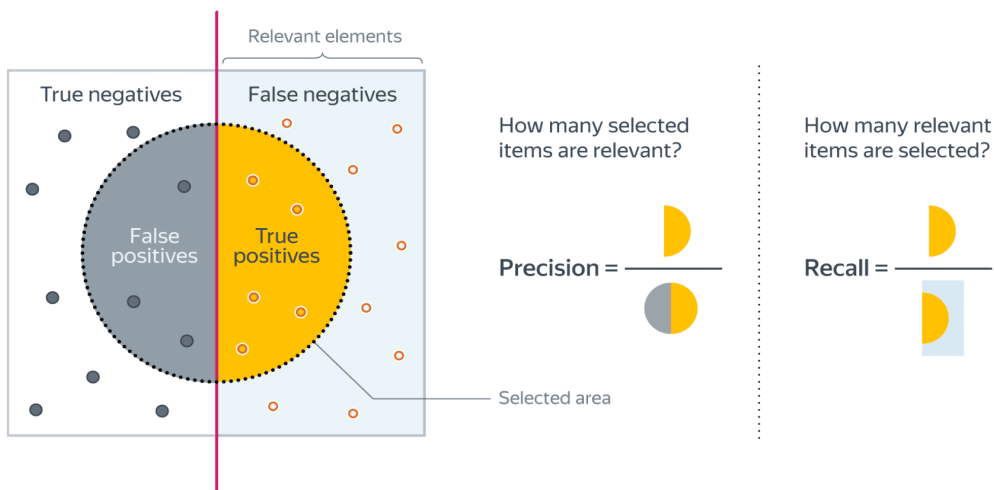
$$\text{error rate} = \frac{FP + FN}{TP + TN + FP + FN}$$

Доля объектов, попавших на главную диагональ — это **accuracy**:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Метрики оценки правильности

Хорошее понимание происходящего даёт следующая картинка:



В случае пары Precision-Recall существует популярный способ скомпоновать их в одну метрику - взять их среднее гармоническое. Данный показатель эффективности исторически носит название **F1-меры (F1-measure)**.

- В случае ассиметрии классов, можно использовать метрики, которые не учитывают TN и ориентируются на TP.
- Если мы рассмотрим долю правильно предсказанных положительных объектов среди всех объектов, предсказанных положительным классом, то мы получим метрику, которая называется **точностью (precision)**

$$\text{Precision} = TP / TP + FP$$

Интуитивно метрика показывает долю релевантных документов среди всех найденных классификатором. Чем меньше ложноположительных срабатываний будет допускать модель, тем больше будет её **Precision**.

Если же мы рассмотрим долю правильно найденных положительных объектов среди всех объектов положительного класса, то мы получим метрику, которая называется **полнотой (recall)**

$$\text{Recall} = TP / TP + FN$$

- Интуитивно метрика показывает долю найденных документов из всех релевантных. Чем меньше ложно отрицательных срабатываний, тем выше **recall** модели.

$$F_1 = \frac{TP}{TP + \frac{FP + FN}{2}}$$